

V1 ROBOT DEVELOPMENT AND VALIDATION PLAN

Autonomous Semantic Navigation Robot

Jan Cuesta

Isaac Sim + ROS 2 Jazzy + Nav2

Version 2.0 — Final planning baseline — 21 September 2026

Purpose

Build and validate an indoor autonomous navigation system in simulation, then package it for controlled integration with the ASU robot. The robot must start reliably, load a saved map, localize, accept a destination, and reach it autonomously—or stop safely and report why it cannot.

This plan follows an iterative systems-engineering workflow: needs and requirements, architecture, implementation, integration, verification, validation, and transition. Every work step advances only after its exit check passes.

Plan control and completion boundary

V1 includes: simulated differential-drive motion, a 2D LiDAR, wheel-derived odometry, mapping, saved-map localization, Nav2 navigation, a C++ mission manager, fault handling, logging, repeatable testing, and a documented interface for V2.

V1 excludes: RGB-D object recognition, semantic destination selection, manipulation, unbounded dynamic crowds, and proof of physical safety. Those require later project phases and real-hardware testing.

Milestone	Required evidence
Exploratory prototype	Isaac Sim, a lightweight mobile base, and basic movement are technically feasible.
Working demo	The robot maps a world, reloads the map, localizes, and reaches selected goals.
V1 complete	Behavior is repeatable; code, faults, startup, tests, evidence, and interfaces are documented.
V2 validated	Real sensors, motors, calibration, stopping behavior, timing, and physical navigation trials pass.

Working rule: an attractive demonstration is not a release. Advance when the exit check passes, retain failed results, and place code, configuration, maps, scene versions, and test evidence under version control.

Development lifecycle

Stage	Purpose	Work steps
01 Definition	Establish intended use, requirements, risks, and hardware assumptions.	1–4
02 Architecture	Define interfaces, ownership, repository, and continuous quality controls.	5–6
03 Simulation	Build the world and integrate motion, sensors, frames, time, and odometry.	7–10
04 Autonomy	Create the map, localize, navigate, and execute missions.	11–14
05 Resilience	Handle faults, evaluate estimation, and automate evidence collection.	15–17
06 Validation	Freeze, test, correct, release, and prepare staged hardware integration.	18–20

Iteration rule: the stages describe dependency order, not a one-pass waterfall. Failed tests return the work to the relevant requirement, model, interface, or implementation step. Risks, requirements, testing, documentation, and configuration control continue throughout the project.

01 / DEFINITION AND GOVERNANCE

Needs before construction

1. Write the Concept of Operations and freeze V1 scope

- ☐ Define operator, intended indoor use, startup, shutdown, mapping, saved-map navigation, goal entry, cancellation, recovery, and manual-stop workflows.
- ☐ Select one differential-drive simulation model, wheel odometry, one 2D LiDAR, and one lightweight test world. Record why the choices transfer to the intended ASU robot.
- ☐ Define world geometry and difficulty: rooms, corridor, doorway widths, static obstacles, temporary blockage, start poses, and at least ten fixed goals.
- ☐ Allow manual driving while mapping; require autonomous motion during navigation acceptance trials.
- ☐ Record exclusions and assumptions, including floor, people, obstacle motion, network, lighting, and manual supervision.

Exit check: A concise ConOps lets another person explain how V1 is operated, what it must do, and what it intentionally does not do.

2. Baseline measurable requirements, traceability, and risks

- ☐ Give every requirement an identifier and a measurable “shall” statement; classify it as functional, performance, interface, safety, or evidence.
- ☐ Create a traceability table linking each requirement to its design component, verification method, test case, and final result.
- ☐ Freeze definitions of success, failure, collision, timeout, manual intervention, acceptable pose error, and stopping response.
- ☐ Establish timing and performance budgets: physics rate, real-time factor, topic rates, maximum data age, command latency, startup time, trial timeout, and sustained duration.
- ☐ Create a lightweight risk register covering uncontrolled motion, stale commands, localization loss, stale sensors, incorrect geometry, simulation slowdown, thermal throttling, and version incompatibility.

Exit check: Every V1 requirement is testable and every major risk has an owner, mitigation, trigger, and planned test.

Initial requirement register

ID	Requirement summary	Primary evidence
NAV-01	Reach valid goals autonomously on the saved map.	30-trial campaign
LOC-01	Localize after a clean restart from three selected starts.	Map-reuse tests
SAFE-01	Stop predictably when velocity commands expire.	Command-loss test
SAFE-02	Do not collide during the frozen acceptance set.	Collision log/video
PERF-01	Maintain the frozen real-time-factor and topic-rate limits.	Performance log
ACC-01	Meet frozen position and heading error tolerances.	Ground-truth comparison
DIAG-01	Report clear mission success, cancellation, timeout, or failure.	Mission logs/tests

3. Confirm the physical robot interface

- ☐ Obtain the exact ASU robot model, onboard computer, operating system, ROS distribution, access method, and responsible contact.
- ☐ Record footprint, wheel radius/separation, maximum speeds, LiDAR/IMU models, sensor mounting frames, and available odometry.
- ☐ Confirm velocity topic and message type, real drivers, command watchdog, emergency/manual stop, power-up behavior, and motor enable sequence.
- ☐ Decide which processes run onboard and which run on the laptop; document expected network, clock, and QoS behavior.
- ☐ Mark unknown values as provisional rather than silently treating simulation values as hardware facts.

Exit check: The intended simulation-to-hardware contract is known, and every unresolved mismatch is explicitly listed.

4. Establish a reproducible development environment

- ☐ Record Ubuntu, NVIDIA driver, Isaac Sim, ROS 2 Jazzy, Nav2, SLAM Toolbox, RViz, compiler, CMake, and colcon versions.
- ☐ Confirm that the selected Isaac Sim release and ROS 2 workflow are compatible; freeze working versions during the sprint.
- ☐ Build and run a small C++ ROS 2 node; verify bidirectional ROS communication with Isaac Sim.
- ☐ Document terminal sourcing, build, test, launch, shutdown, and clean-start commands.
- ☐ Retain the 30-FPS safe launcher as the development baseline; treat viewport FPS separately from physics and real-time factor.

Exit check: A clean terminal can reproduce the build and communicate with the simulator using documented commands.

02 / ARCHITECTURE AND PROJECT FOUNDATION**Interfaces before integration****5. Define system architecture and interface control**

- ☐ Draw the node/component architecture for simulator, robot base, sensors, localization, Nav2, mission manager, logging, and safety timeout.
- ☐ Create an interface table for topics, actions, services, message types, frame IDs, expected rates, QoS, time source, and publisher ownership.
- ☐ Define the TF tree before sensor integration: `map -> odom -> base_link -> sensors`, with documented platform-specific intermediate frames.
- ☐ Assign exactly one publisher to each dynamic transform and one controlled path for motion commands.
- ☐ Define which simulated components are replaced by hardware drivers in V2 without changing upper-level navigation interfaces.

Exit check: The ROS graph, TF ownership, command path, and simulation/hardware replacements are unambiguous before implementation.

6. Create the repository, packages, and continuous quality controls

- ☐ Create the workspace under `/projects`; initialize Git with README, license decision, issue list, change log, and tagged baselines.
- ☐ Organize the description, simulation, bringup, navigation, mission, and test responsibilities as the six ROS packages defined in the architecture.
- ☐ Store parameters in versioned configuration files; organize maps, worlds, launch files, mission files, results, and documentation.
- ☐ Enable `ament_lint`; add unit tests with the first production code and integration tests with the first cross-package interface.
- ☐ Establish the repeatable quality command: build without warnings, run `colcon test`, inspect results, and perform a clean-start smoke test.
- ☐ Exclude generated builds and large bags; document how large simulator assets are retrieved.

Exit check: The project rebuilds from source, package responsibilities are clear, and automated checks run from the beginning.

03 / SIMULATION AND COMPONENT INTEGRATION**Build the minimum valid system****7. Build and version the lightweight Isaac Sim acceptance world**

- ☐ Build the frozen floor, walls, corridor, doorways, localization features, and obstacle set using simple geometry and collision shapes.
- ☐ Verify dimensions, robot clearance, friction, spawn poses, lighting, collision geometry, and obstacle mobility required by tests.
- ☐ Use a mobile base approximating the physical platform; remove cameras, decorative assets, and unused sensor pipelines from V1.
- ☐ Save the scene and dependencies; close Isaac Sim, reopen the stage, and verify the same configuration loads.
- ☐ Establish a performance baseline for world plus robot before sensors and navigation are added.

Exit check: The complete scene reloads correctly, matches the frozen test geometry, and runs without prolonged stalls.

8. Connect motion to ROS and enforce command expiry

- ☐ Connect ROS 2 velocity commands to the differential-drive controller using the interface contract.
- ☐ Test forward, reverse, left rotation, and right rotation independently; verify axes, units, wheel order, radius, and separation.
- ☐ Apply conservative velocity, acceleration, and deceleration limits appropriate to the acceptance-world clearance.
- ☐ Maintain one controlled command path and prevent competing publishers from commanding the base unexpectedly.
- ☐ Implement controller/base-level command expiry and verify predictable stopping after command loss.

Exit check: Motion matches commands and stale, absent, or terminated command sources produce the documented stop response.

9. Integrate sensors, simulation time, and transforms

- ☐ Publish 2D LiDAR, wheel/joint measurements, odometry, simulation clock, and sensor transforms; add IMU data only if required for the V2 interface or planned comparison.
- ☐ Validate timestamps, simulation-time use, message rates, maximum data age, QoS compatibility, and publisher ownership.
- ☐ Verify in RViz that laser returns align with walls, sensor frames are correctly located, odometry direction is correct, and TF has no recurring errors.
- ☐ Measure performance again after each sensor or bridge addition; record the change in real-time factor, CPU/GPU load, temperatures, and topic rates.

Exit check: The moving robot produces coherent, timely sensor data consistent with the architecture and performance budgets.

10. Validate transfer-oriented wheel odometry

- ☐ Identify whether any Isaac example publishes perfect simulator pose; label it ground truth rather than odometry.
- ☐ Use ground truth only for early debugging and independent error measurement; derive navigation odometry from simulated wheel motion.
- ☐ Test straight lines, rotation, and a square trajectory; measure translation and heading errors against ground truth.
- ☐ Repeat with controlled wheel-radius error, wheel-separation error, slip/noise, or bias appropriate to the risk plan.
- ☐ Document the source, covariance assumptions, update rate, and transform ownership of every pose estimate.

Exit check: Navigation does not depend on perfect global simulator pose, and odometry performance is measured and documented.

04 / MAPPING, LOCALIZATION, AND AUTONOMY

Integrate only after components pass

11. Create and save the occupancy map

- ☐ Configure SLAM Toolbox using the validated sensor, odometry, TF, and time interfaces.
- ☐ Drive slowly through the world, complete useful loops, and inspect scan alignment continuously.
- ☐ Investigate duplicated walls, distortion, discontinuities, and pose jumps before accepting the map.
- ☐ Save the occupancy map and metadata; save the SLAM session separately only if mapping continuation is required.
- ☐ Version the world, robot configuration, SLAM parameters, and map together.

Exit check: A usable map has been produced from simulated robot sensors and survives a clean restart.

12. Localize on the saved map after restart

- ☐ Start saved-map navigation mode, load the map, and use AMCL for localization.
- ☐ Supply an approximate initial pose and test at least three frozen starting positions and orientations.
- ☐ Ensure that only the selected mapping or localization component publishes `map -> odom`.
- ☐ Block mission acceptance until localization health meets the defined criterion.
- ☐ Test an intentionally incorrect initial pose and document detection, recovery, or required operator action.

Exit check: The robot reliably localizes on the existing map without rebuilding it and reports when localization is not trustworthy.

13. Configure and tune autonomous navigation

- ☐ Configure robot footprint, global and local costmaps, obstacle layers, inflation/clearance, planner, controller, and behavior plugins.
- ☐ Set velocity/acceleration limits, goal and progress checkers, recovery behavior, lifecycle startup, and mission readiness checks.
- ☐ Progress through open-space goals, turns, doorways, corridors, and multi-room routes.
- ☐ Correct geometry, sensing, timing, and TF faults before compensating through planner/controller tuning.
- ☐ Record each relevant tuning change and rerun affected routes to detect regressions.

Exit check: Representative routes complete repeatedly without manual steering, collision, or unexplained recovery loops.

14. Develop the C++ mission manager

- ☐ Load named waypoints or a mission file, validate inputs, and wait for localization and Nav2 readiness.
- ☐ Send goals through a ROS action client; track feedback, acceptance, rejection, completion, cancellation, and failure.
- ☐ Implement explicit `idle`, `preparing`, `navigating`, `succeeded`, `failed`, and `cancelled` states.
- ☐ Apply frozen timeouts, bounded retries, status reporting, and result logging.
- ☐ Unit-test mission parsing and state transitions; integration-test rejection, cancellation, timeout, and retry limits.

Exit check: A predefined mission executes, cancels safely, and produces a clear, testable final result.

05 / RESILIENCE, ESTIMATION, AND EVIDENCE**Demonstrate failure behavior****15. Implement fault handling and safe stopping**

- ☐ Convert the risk register into explicit responses for stale LiDAR, odometry, TF, localization loss, blocked paths, unreachable goals, and overheating/performance limits.
- ☐ Inject command loss, mission-process exit, Nav2 failure, simulator pause/reset, and selected sensor/transform failures.
- ☐ Specify which conditions block startup, cancel a mission, stop motion, attempt bounded recovery, or require explicit restart.
- ☐ Keep command-expiry protection at the base/controller level; do not depend solely on the mission process remaining alive.
- ☐ Retain diagnostic evidence for each injected fault and verify the observed response against the requirement.

Exit check: Injected faults trigger the intended stop, recovery, or failure response without uncontrolled motion.

16. Evaluate sensor fusion only when justified

- ☐ Confirm whether V2 requires locally configured wheel-odometry/IMU fusion or already provides an equivalent estimate.
- ☐ If required, configure `robot_localization` EKF; verify IMU axes, units, covariances, duplicated information, and transform ownership.
- ☐ Compare wheel-only and fused estimates on identical trajectories using independent simulator ground truth.
- ☐ Document improvements, regressions, parameter sensitivity, and the recommended V2 configuration.
- ☐ Defer fusion if it adds no requirement-backed value; record the decision rather than adding complexity by default.

Exit check: Measured evidence supports either the proposed fusion configuration or a documented decision to defer it.

17. Complete automated system testing, logging, and metrics

- ☐ Record scenario, start pose, goal, code/configuration versions, success, completion time, final pose error, and final mission state.
- ☐ Log collisions, interventions, recoveries, sensor/TF faults, latency, topic rates, real-time factor, resource use, and thermal results.
- ☐ Retain representative rosbag runs and important failures; separate true-pose error from estimated-pose metrics.
- ☐ Automate end-to-end simulation scenarios while preserving unit and integration tests created throughout development.
- ☐ Produce a requirement-to-test report showing that every frozen requirement has evidence or an explicit open disposition.

Exit check: Scenarios are repeatable, results are comparable across changes, and requirement coverage is visible.

06 / ACCEPTANCE, RELEASE, AND HARDWARE TRANSITION

Freeze before judging

18. Run the frozen acceptance campaign

- ☐ Freeze code, world, map, parameters, tolerances, timeouts, sensor-age limits, and stopping responses before official testing.
- ☐ Perform three clean cold starts and saved-map localization from three selected start poses.
- ☐ Run ten fixed navigation goals three times, covering turns, doorways, corridors, and multi-room routes.
- ☐ Test temporary blockage, unreachable goals, changed obstacles, imperfect odometry, and sensor/command loss.
- ☐ Run a sustained session and retain all failures even when the aggregate success target passes.

Exit check: Every frozen requirement has a pass/fail result with traceable supporting evidence.

19. Investigate defects and perform regression testing

- ☐ Prioritize uncontrolled motion, collisions, localization loss, crashes, startup failures, and timing violations.
- ☐ Reproduce important failures; identify the violated requirement and plausible root cause.
- ☐ Change one relevant factor at a time, document the rationale, and rerun the affected component/integration tests.
- ☐ Rerun the final nominal campaign against the frozen release candidate.
- ☐ Document at least two meaningful failure investigations and retain known limitations.

Exit check: No blocking defect remains, fixes have regression evidence, and accepted limitations are explicit.

20. Package the release and stage the V2 handoff

- ☐ Provide separate simulation and hardware launch configurations with documented topic, message, frame, QoS, timing, and parameter contracts.
- ☐ Remove simulation-only publishers from hardware launch; configure real-system time and clearly mark every unverified hardware value.
- ☐ Include architecture, dependencies, startup/shutdown, maps, scene assets, mission procedures, calibration guidance, test evidence, demo video, and known limitations.
- ☐ Tag the V1 code/configuration release and preserve the exact acceptance baseline.
- ☐ Prepare the staged physical-integration checklist on the following page; do not proceed directly from simulation to autonomous driving.

Exit check: The V1 release is reproducible and ready for controlled physical integration, with remaining hardware verification clearly identified.

Acceptance matrix

The values below are project targets, not universal robotics standards. Confirm them during Step 2 against robot speed, sensor rates, available clearance, and ASU hardware constraints; then freeze them before Step 18.

Test	Requirement	Acceptance evidence
Cold startup	START-01	Three clean starts without undocumented fixes.
Map reuse	LOC-01	Saved map loads and localization succeeds from three chosen starts.
Nominal navigation	NAV-01	Ten fixed goals repeated three times; initial target at least 27/30 successes without intervention.
Collision	SAFE-02	Zero observed collisions in the acceptance set; any collision blocks release until investigated and dispositioned.
Goal accuracy	ACC-01	Initial target: position error at most 0.20 m and heading error at most 10 degrees, measured against independent ground truth.
Temporary blockage	NAV-02	Robot waits or replans and completes when a feasible route remains.
Unreachable goal	DIAG-01	Robot stops and reports failure within the frozen timeout.
Command loss	SAFE-01	Controller-level expiry produces the frozen stop response.
Sensor/TF loss	SAFE-03	Detection, cancellation/stop behavior, and diagnostic output meet the frozen requirement.
Performance	PERF-01	Real-time factor, topic rates, data age, latency, and thermal thresholds remain within the frozen budget.
Robustness	ROB-01	Changed obstacles and imperfect odometry are tested; observed limitations are retained.
Sustained operation	PERF-02	Thirty-minute session without unexplained crash, accumulating fault, or timing degradation.

Staged V2 physical-integration gate

- ☐ Confirm emergency stop, manual stop, power isolation, safe test area, supervision, and low-speed limits.
- ☐ Verify real topics, timestamps, TF, LiDAR/IMU data, and network behavior with motors disabled.
- ☐ Verify left/right motor direction and encoder sign with the platform safely lifted or otherwise restrained according to lab procedure.
- ☐ Test command expiry and shutdown behavior before free motion.
- ☐ Run very-low-speed forward, reverse, and rotation tests in an empty controlled area.
- ☐ Calibrate wheel radius/separation, footprint, sensor extrinsics, and odometry/IMU behavior.
- ☐ Validate stopping distance and clearance assumptions at the approved test speed.
- ☐ Map a small controlled area, restart, and localize without autonomous motion.
- ☐ Execute short open-area goals; progress later to corridors, doorways, and controlled obstacles.
- ☐ Run at least 20 physical trials only after prior gates pass; initial target at least 90% completion without intervention and zero uninvestigated collisions.

Important: V1 simulation evidence does not validate physical braking, wheel slip, motor polarity, sensor calibration, networking, human interaction, or emergency-stop effectiveness.

Final release checklist

- | | |
|--|--|
| ☐ Tagged code and pinned versions | ☐ Reproducible build/test/launch instructions |
| ☐ Baselined requirements and traceability | ☐ Architecture and interface-control tables |
| ☐ Saved map and complete scene dependencies | ☐ Unit, integration, and system test results |
| ☐ Acceptance trials and representative bags | ☐ Two documented failure investigations |
| ☐ Performance and thermal evidence | ☐ Demo video and mission result examples |
| ☐ Risk register and known limitations | ☐ Staged V2 physical-integration checklist |

Technical basis

This project-scale plan adapts established engineering practice rather than claiming formal certification. The following sources support its lifecycle and technical ordering:

- NASA, [Systems Engineering Handbook](#): stakeholder expectations, requirements, design, implementation, integration, verification, validation, transition, and continuous technical management.
- Nav2, [First-Time Robot Setup Guide](#): transforms, robot description, odometry, perception sensors, mapping/localization, footprint, navigation plugins, and lifecycle bringup.
- ROS 2 Jazzy, [Developer Guide](#) and [Quality Guide](#): documentation, unit/integration/system testing, linting, static analysis, and repeatable quality practices.
- NVIDIA Isaac Sim, [ROS 2 Navigation](#): Isaac Sim, occupancy maps, Nav2, ROS actions, and programmatic goal execution.

Project decision

Isaac Sim replaces the older Gazebo assumption for V1. The simulator-specific bridge and model may change; the requirements, interfaces, tests, and navigation behavior remain the controlling project baseline.